BITS Pilani
K K Birla Goa Campus

INSTRUCTION SET

MICROPROCESSORS & INTERFACING (CS F241)

Dr. Gargi Alavani
Department of CS & IS

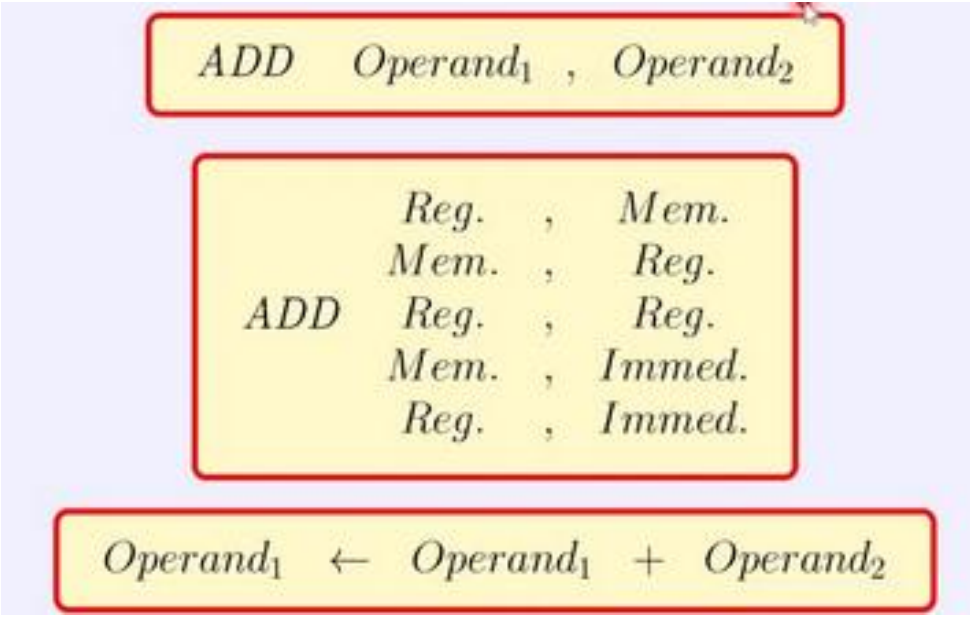
innovate achieve lead

Example 3

What is the machine code for

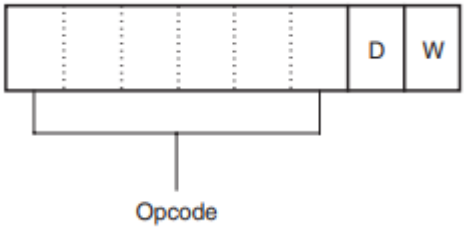
ADD [BX+DI+1234H], AX

Opcode for ADD is 000000.

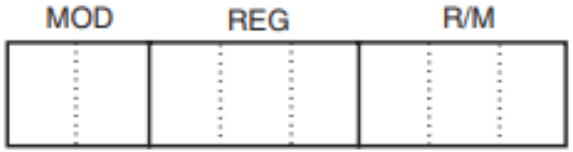


Output of ADD instruction will be stored in memory address hence D=0. AX is source operand (REG). Since data is moving from REG, D=0.

Example 3: ADD [BX+DI+1234H], AX



Byte 1



Byte 2

MOD	*Function*
00	No displacement
01	8-bit sign-extended displacement
10	16-bit signed displacement
11	R/M is a register

Code	W = 0 *(Byte)*	W = 1 *(Word)*	W = 1 *(Doubleword)*
000	AL	AX	EAX
001	CL	CX	ECX
010	DL	DX	EDX
011	BL	BX	EBX
100	AH	SP	ESP
101	CH	BP	EBP
110	DH	SI	ESI
111	BH	DI	EDI

MOD / R/M	00	01	10	11	
				W = 0	W = 1
000	[BX] + [SI]	[BX] + [SI] + d8	[BX] + [SI] + d16	AL	AX
001	[BX] + [DI]	[BX] + [DI] + d8	[BX] + [DI] + d16	CL	CX
010	[BP] + [SI]	[BP] + [SI] + d8	[BP] + [SI] + d16	DL	DX
011	[BP] + [DI]	[BP] + [DI] + d8	[BP] + [DI] + d16	BL	BX
100	[SI]	[SI] + d8	[SI] + d16	AH	SP
101	[DI]	[DI] + d8	[DI] + d16	CH	BP
110	d16 (direct address)	[BP] + d8	[BP] + d16	DH	SI
111	[BX]	[BX] + d8	[BX] + d16	BH	DI
	MEMORY MODE			REGISTER MODE	

d8 = 8-bit displacement d16 = 16-bit displacement

Example 3

- What is the machine code for ADD [BX+DI+1234H], AX ?

Opcode=000000
D=0 since AX is source operand
W=1 ->16 bit data operation
Byte1=(00000001) = 01H
MOD= 10 (Memory mode with 16-bit displacement)
REG=000 (Code of AX)
R/M=001 (BX+DI)
Byte2=(10000001)=81H
Displacement=1234H -> stored as LSB MSB
Machine code = 01813412

Example 4

What is the machine code for MOV CS:[BX], DL ?

- In this instruction, CS: indicates Physical Address =BX+CS*10H
- **CS: Segment Override Prefix**
- Code byte for Segment Override Prefix = **001XX110** which forms the **first byte of instruction**.
- XX-> ES=00, CS=01, SS=10, DS=11

Example 3

Byte 1 = 00101110

Opcode =100010

D= from REG = 0

W= 0

Byte 2 = 10001000

MOD=00 (Memory, No Displacement)

REG=010 (DL)

R/M = 111 ([BX])

Byte 3= 00010111

Data Movement Instructions

MOV Instructions available:

- MOV
- MOVSX -**Move with Sign-Extend**
- MOVZX -**Move with Zero-Extend**

- When do you use MOVSX and MOVZX?

- Need to work with values of different sizes (e.g., moving from a byte to a word) while preserving the sign (for MOVSX) or not (for MOVZX).

Data Movement Instructions

MOVSX -**Move with Sign-Extend**

- This instruction copies the source operand into the destination operand, while sign-extending the source to fill the destination.

e.g. MOVSX AX, AL

MOVZX -**Move with Zero-Extend**

- Copies the source operand into the destination operand. However, in this case, zero-extension is performed instead of sign-extension.

e.g. MOVZX AX, AL

Stack Memory Instructions

Two Instructions

- PUSH
- POP

Six forms of PUSH and POP instructions: register, memory, immediate, segment register, flags and all registers.

Note that PUSH and POP immediate and PUSHA and POPA (all registers) are not available in 8086, but are available 80286 onwards.

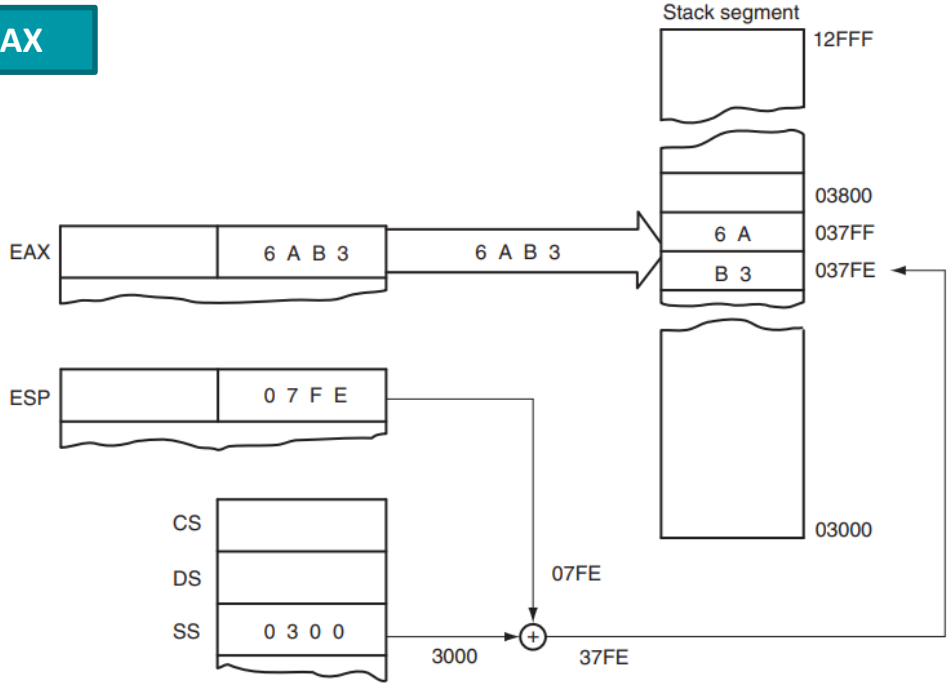
PUSH Instruction

- Transfers 2 bytes of data to the stack-> **PUSH** and **POP** work with 16 bit values only!
- Source: any internal 16 bit register, immediate data(not in 8086), any segment register, any 2 bytes of memory data
- When data are pushed onto the stack, the first (most-significant) data byte moves to the stack segment memory location addressed by SP-1
- Second data byte moves to SP-2
- After the two PUSH operations, contents of SP= decrement by 2 **SP = SP-2**

The 8086 architecture was designed with a focus on 16-bit data processing, and as a result, the stack-related instructions, including push and pop, work with 16-bit registers. Attempting to use these instructions directly with **8-bit registers** would result in **unexpected behavior or errors**.

PUSH Instruction

PUSH AX



Endianness (Byte Order)

Endianness defines **which byte of a number is stored first in memory**.

Little Endian

- In **Little Endian** format, the **least significant byte (LSB)** is stored at the **lowest memory address**, and the **most significant byte (MSB)** is stored at the **highest memory address**.
- This format prioritizes storing the **"little end" (the smallest value)** first.

Example:

- For the 32-bit hexadecimal number 0x12345678:
- Memory layout (from lowest to highest address):
  78 56 34 12
- Commonly used by:
  - Intel x86 and x86-64 architectures.
  - ARM processors in Little Endian mode.

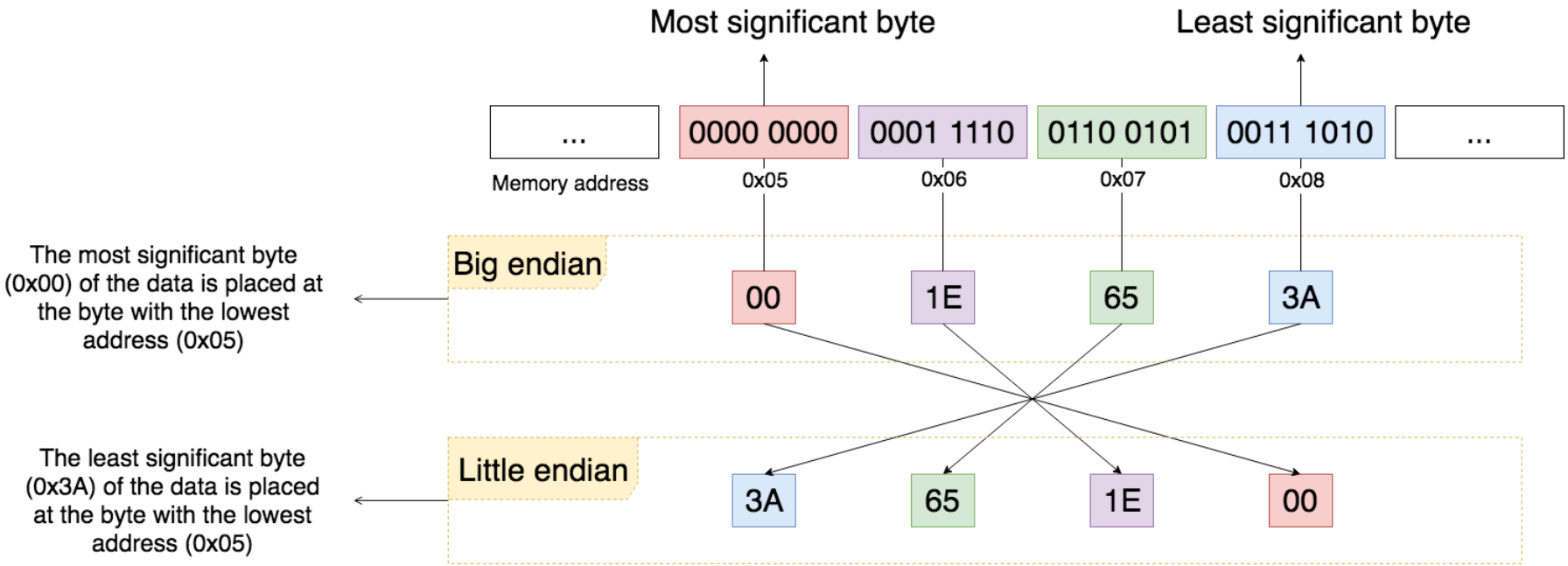
Big Endian

- In **Big Endian** format, the **most significant byte (MSB)** is stored at the **lowest memory address**, and the **least significant byte (LSB)** is stored at the **highest memory address**.
- This format prioritizes storing the **"big end" (the largest value)** first.

Example:

- For the 32-bit hexadecimal number 0x12345678:
- Memory layout (from lowest to highest address):
  12 34 56 78
- Commonly used by:
  - PowerPC
  - Some network protocols (e.g., TCP/IP) that specify Big Endian for transmitting data

Big Endian vs Little Endian



PUSH Instructions

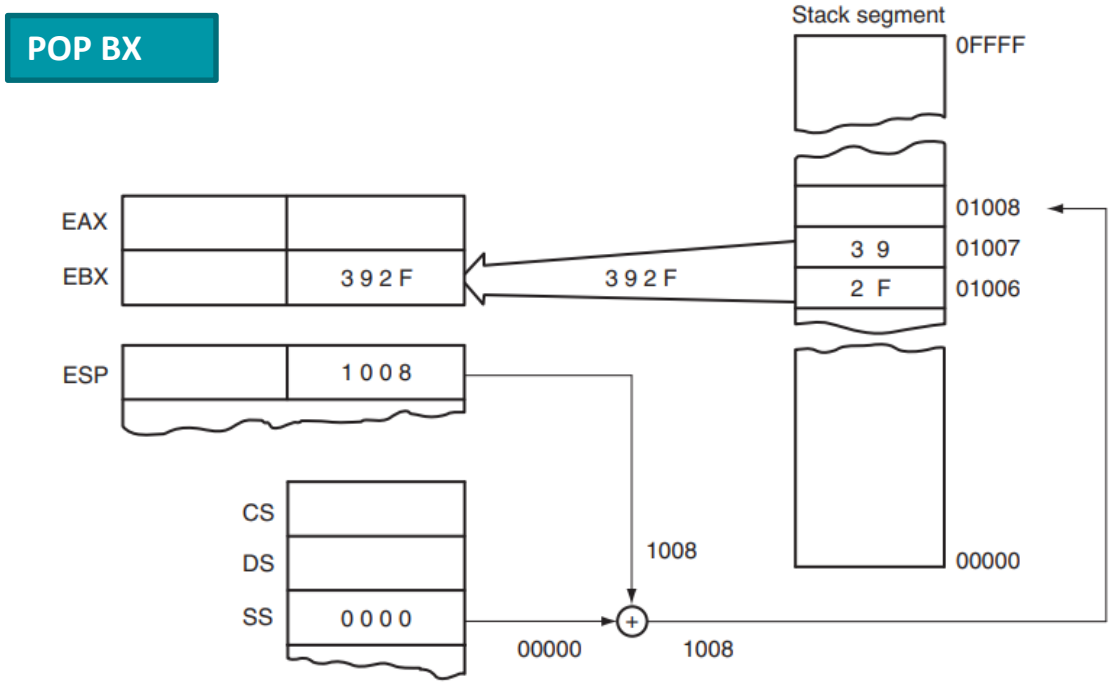
Symbolic	*Example*	*Note*
PUSH reg16	PUSH BX	16-bit register
PUSH reg32	PUSH EDX	32-bit register
PUSH mem16	PUSH WORD PTR[BX]	16-bit pointer
PUSH mem32	PUSH DWORD PTR[EBX]	32-bit pointer
PUSH mem64	PUSH QWORD PTR[RBX]	64-bit pointer (64-bit mode)
PUSH seg	PUSH DS	Segment register
PUSH imm8	PUSH 'R'	8-bit immediate
PUSH imm16	PUSH 1000H	16-bit immediate
PUSHD imm32	PUSHD 20	32-bit immediate
PUSHA	PUSHA	Save all 16-bit registers
PUSHAD	PUSHAD	Save all 32-bit registers
PUSHF	PUSHF	Save flags
PUSHFD	PUSHFD	Save EFLAGS

POP Instruction

- Performs the inverse operation of a PUSH instruction
- **Removes data from the stack** and places it into 16-bit target register, segment register, or a 16 bit memory location
- **POPF**-removes 16-bit numbers from stack and places it into the **flags register**

SP = SP+2

POP Instruction



POP Instructions

Symbolic	*Example*	*Note*
POP reg16	POP CX	16-bit register
POP reg32	POP EBP	32-bit register
POP mem16	POP WORD PTR[BX+1]	16-bit pointer
POP mem32	POP DATA3	32-bit memory address
POP mem64	POP FROG	64-bit memory address (64-bit mode)
POP seg	POP FS	Segment register
POPA	POPA	Pops all 16-bit registers
POPAD	POPAD	Pops all 32-bit registers
POPF	POPF	Pops flags
POPFD	POPFD	Pops EFLAGS

LEA- LOAD-EFFECTIVE ADDRESS

- The LEA instruction loads a 16- or 32-bit register with the offset address of the data specified by the operand.

E.g. LEA AX,NUMB

- Loads AX with the offset address of NUMB

- MOV BX,OFFSET LIST is same as LEA BX,LIST

Why is the LEA instruction available if the OFFSET directive accomplishes the same task?

- OFFSET only functions with simple operands such as LIST. It may not be used for an operand such as [DI], LIST [SI], and so on.
- It takes the microprocessor longer to execute the LEA BX,LIST instruction than the MOV BX,OFFSET LIST.
- It is because the **assembler calculates the offset address of LIST**, whereas the microprocessor calculates the address for the LEA instruction.
- The MOV BX,OFFSET LIST instruction is actually assembled as a move immediate instruction and hence is more efficient.

Example

E.g. LEA BX, [DI]
DI=1000H -> BX=1000H

Can this be done using MOV BX, DI ?

E.g. LEA SI,[BX+DI]
BX=3000H, DI=2000H -> SI=3000H (Sum= modulo-64)

If BX=1000H DI = FF00H , what is SI?

Example

E.g. LEA BX, [DI]
DI=1000H -> BX=1000H

Can this be done using MOV BX, DI ?

E.g. LEA SI,[BX+DI]
BX=3000H, DI=2000H -> SI=3000H (Sum= modulo 2^{16})

If BX=1000H DI = FF00H , what is SI?

LEA BX, [DI] is same as MOV BX, DI
Both transfer a copy of offset address(DI) in BX

DI – Data stored at DI
[DI] – Data stored at the address of DI

Example on DosBox

```
-A 100
0859:0100 MOV DI,0101
0859:0103 MOV AX,DI
0859:0105 LEA BX,[DI]
0859:0107
-T
AX=0000 BX=0000 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=0859 ES=0859 SS=0859 CS=0859 IP=011E NV UP EI NG NZ NA PO NC
0859:011E 0000              ADD     [BX+SI],AL                      DS:0000=CD
-T=100
AX=0000 BX=0000 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0101
DS=0859 ES=0859 SS=0859 CS=0859 IP=0103 NV UP EI NG NZ NA PO NC
0859:0103 89F8              MOV     AX,DI
-
AX=0101 BX=0000 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0101
DS=0859 ES=0859 SS=0859 CS=0859 IP=0105 NV UP EI NG NZ NA PO NC
0859:0105 8D1D              LEA     BX,[DI]                     DS:0101=0101
-
AX=0101 BX=0101 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0101
DS=0859 ES=0859 SS=0859 CS=0859 IP=0107 NV UP EI NG NZ NA PO NC
0859:0107 89F8              MOV     AX,DI
-
```

Example

```
        .DATA                   ;start data segment
DATA1   DW      2000H           ;define DATA1
DATA2   DW      3000H           ;define DATA2
        .CODE                   ;start code segment
        .STARTUP                ;start program
        LEA SI,DATA1            ;address DATA1 with SI
        MOV DI,OFFSET DATA2     ;address DATA2 with DI
        MOV BX,[SI]             ;exchange DAT1 with DATA2
        MOV CX,[DI]
        MOV [SI],CX
        MOV [DI],BX
        .EXIT
        END
```

BITS Pilani
K K Birla Goa Campus

Thank You